

Лабораторная работа №3

Обработка границ, детектирование прямых и отрезков

Теоретическая часть

Введение

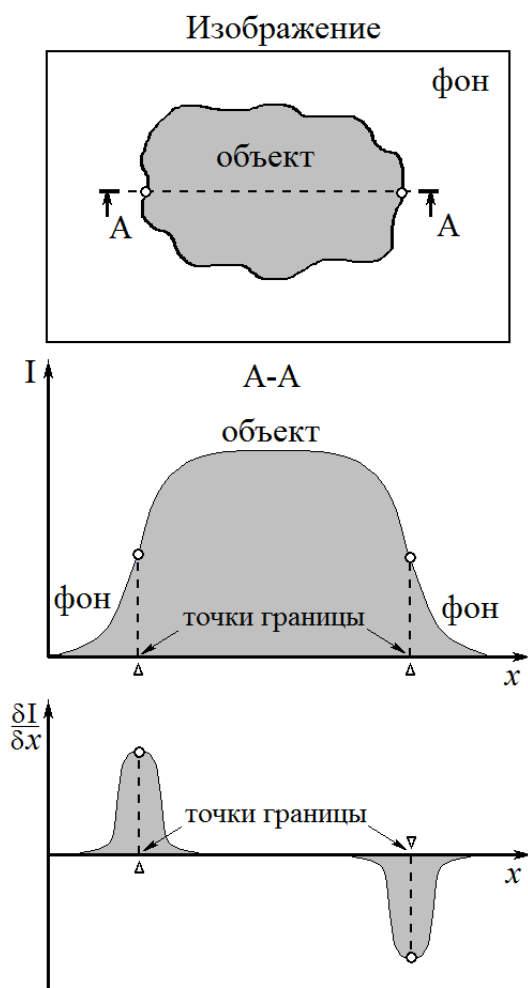
На прошлой лабораторной работе мы рассматривали случай, когда цвет точек объекта и фона заранее известен, и его точки могут быть отделены от фона, чтобы затем можно было найти и распознать объект морфологическими методами.

Однако цвет объекта известен не всегда. Более того, в ряде задач объект специально окрашен в цвета, максимально близкие к цвету фона. Что же делать в этом случае?

Если объект физически отличается от окружающего фона, то при переходе от точек объекта к точкам фона яркость изменится скачком: в этом месте и находятся границы объекта. Таким образом, мы все равно имеем возможность найти границы объекта по перепаду яркости, и представить объект в виде совокупности точек его границ. Более того, точки границы можно собирать в более крупные геометрические объекты – прямые, отрезки, окружности и т.д. Такое представление будет обладать следующими свойствами:

- надежность: расчет признаков опирается на большое количество точек;
- компактность описания: вместо сотен тысяч точек используются десятки объектов;
- универсальность: большинство объектов имеют хорошо заметные и легко выделяемые границы.

Выделение точек границ



Пусть на изображении имеется объект, цвет которого заранее неизвестен. Рассмотрим функцию яркости в некотором сечении А-А. Точки границы объекта могут быть найдены как точки, в которых яркость изменяется (возрастает или убывает) наиболее быстро. Скорость изменения яркости – это ни что иное, как производная яркости по координате $\delta I / \delta x$.

Значит, для обнаружения точек границ достаточно найти точки локального экстремума производной яркости.

Для поиска границ с использованием этого подхода нужно решить две проблемы:

1.Изображение – дискретная функция, и производная может принимать значения только 0 и $\pm\infty$. Значит, нужно найти дискретный аналог производной, например – в виде разностной схемы.

2.Изображение –двухмерная функция. Это значит, что искать производную нужно не только по оси x , но и по оси y , и по всем остальным направлениям. Из всех производных в точке по всем возможным направлениям нужно взять максимальную по модулю.

Фильтр Собела

Для построения простейшей разностной схемы можно воспользоваться определением производной:

$$\frac{\partial I(x, y)}{\partial x} = \lim_{\Delta x \rightarrow 0} \frac{I(x + \Delta x, y) - I(x, y)}{\Delta x}$$

Минимальное значение Δx – это шаг сетки, равный единице. Поэтому можно записать приближенное равенство:

$$\frac{\partial I(x, y)}{\partial x} \approx \{\Delta x = 1\} \approx I(x + \Delta x, y) - I(x, y)$$

Аналогично для оси Y:

$$\frac{\partial I(x, y)}{\partial y} \approx \{\Delta y = 1\} \approx I(x, y + \Delta y) - I(x, y)$$

Можно также записать производную в виде маски фильтра. Маска фильтра для производной по x:

$$M_x = \begin{bmatrix} 1 & -1 \end{bmatrix}$$

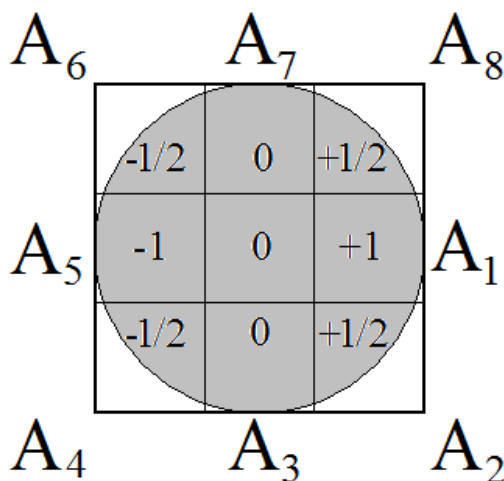
Маска фильтра для производной по y: $M_y = \begin{bmatrix} 1 \\ -1 \end{bmatrix}$

Можно убедиться, что такой простой подход вполне работоспособен, однако имеет существенные недостатки:

- Крайне низкая точность определения направления градиента. Для направлений границ, не кратных 45 градусам, производные вычисляются слишком грубо и не позволяют найти точное направление границы.

- Высокая чувствительность к шуму. Поскольку для вычислений каждой производной используется лишь две точки, то вероятность зашумления хотя бы одной из них очень высока. В результате случайные шумовые точки будут помечены как границы объекта, тогда как реальные границы могут быть пропущены.

- Неизотропность. Результат выделения границ будет различным при различных направлениях прохода по изображению. Более того, само положение точки границы будет смещаться относительно реального положения, причем смещение зависит от направления прохода.



Для преодоления этих недостатков следует учитывать большее количество точек окрестности текущей точки, более точную формулу для замены производной, которая будет изотропна и симметрична относительно текущей точки. Такой фильтр можно получить с использованием *теории кругового оператора*.

Рассмотрим окрестность текущей точки размером 3x3. Впишем в нее окружность диаметром 3 точки. Точки справа и слева (A1 и A5) закроются окружностью целиком, а угловые точки (A2, A4, A6, A8) – только наполовину. Дадим угловым точкам веса $\pm 1/2$, а левой и правой точке – веса -1 и +1 соответственно. Центральной точке A0, верхней и нижней точке (A3 и A7) дадим нулевые веса. Для получения целочисленной маски домножим все веса на 2. Получим маску, которая называется *маской Собела*:

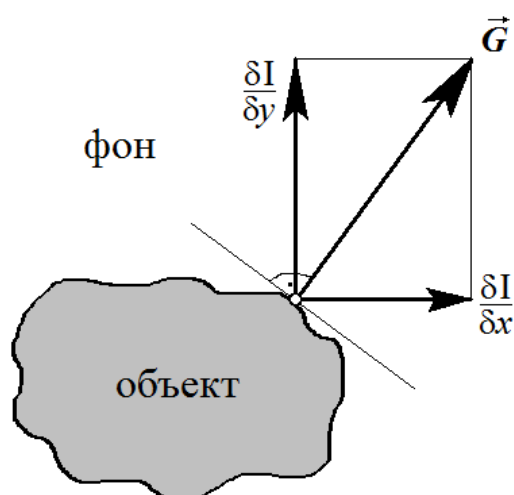
$$S_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}, S_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}$$

Соответственно, формула для вычисления производной будет иметь вид:

$$\frac{\partial I(x, y)}{\partial x} \approx \sum_{\Delta x=-1}^1 \sum_{\Delta y=-1}^1 I[x + \Delta x, y + \Delta y] S_x[\Delta x, \Delta y],$$

$$\frac{\partial I(x, y)}{\partial y} \approx \sum_{\Delta x=-1}^1 \sum_{\Delta y=-1}^1 I[x + \Delta x, y + \Delta y] S_y[\Delta x, \Delta y].$$

Фильтр Собела даёт хорошую точность расчета производных и широко используется на практике. Возможны также фильтры Собела большего размера (5x5, 7x7), которые дают еще большую точность, но и требуют существенно больше вычислений.



Использование градиента яркости

Введем в рассмотрение вектор

$$\vec{G}(x, y) = \left(\frac{\partial I(x, y)}{\partial x}, \frac{\partial I(x, y)}{\partial y} \right), \quad \text{составленный из}$$

частных производных функции яркости изображения в данной точке. Этот вектор представляет собой вектор градиента яркости.

Модуль вектора градиента будет показывать наибольшую скорость изменения яркости, то есть степень выраженности границы в данной точке. Это значит, что нам не надо искать производные по всем возможным направлениям, достаточно производных по x и по y .

Модуль градиента может быть вычислен по формуле:

$$|\vec{G}(x, y)| = \sqrt{\left(\frac{\partial I(x, y)}{\partial x} \right)^2 + \left(\frac{\partial I(x, y)}{\partial y} \right)^2}$$

Направление градиента показывает направление, в котором яркость возрастает наиболее быстро. Это направление всегда перпендикулярно направлению границ, и может быть использовано для его определения. Направление градиента может быть вычислено по формуле:

$$\arg \vec{G} = \arctan \frac{\frac{\delta I}{\delta y}}{\frac{\delta I}{\delta x}}$$

В приложениях, чувствительных к объему вычислений, модуль градиента яркости может быть вычислен по приближенным формулам без использования квадратного корня:

$$|\vec{G}(x, y)| \approx \max \left(\left| \frac{\partial I(x, y)}{\partial x} \right|, \left| \frac{\partial I(x, y)}{\partial y} \right| \right) \quad (\text{низкая точность}),$$

$$|\vec{G}(x, y)| \approx \frac{1}{2} \left[\left| \frac{\partial I(x, y)}{\partial x} \right| + \left| \frac{\partial I(x, y)}{\partial y} \right| + \max \left(\left| \frac{\partial I(x, y)}{\partial x} \right|, \left| \frac{\partial I(x, y)}{\partial y} \right| \right) \right] \quad (\text{средняя точность}).$$

Как правило, точное значение модуля градиента в дальнейших вычислениях не используется (производится только его сравнение с порогом), и такая аппроксимация вполне допустима.

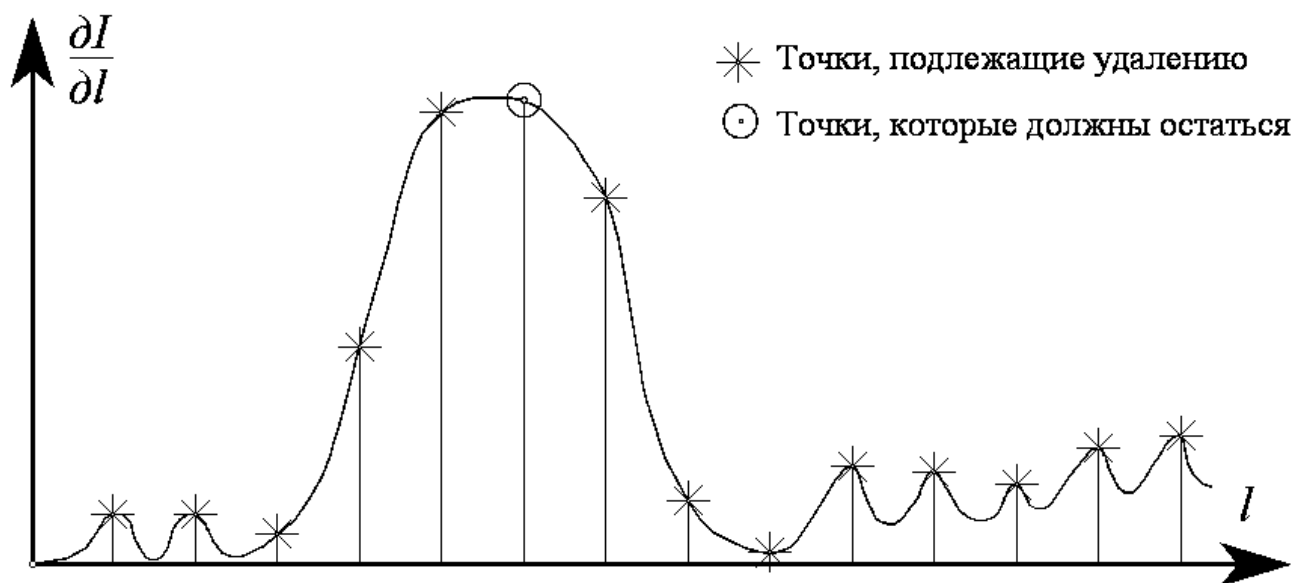
Для вычисления направления градиента удобно использовать функцию $\text{atan2}(y, x)$, которая реализована в большинстве современных языков программирования. Она рассматривает (x, y) как точку и вычисляет угол направления на нее в диапазоне от 0 до 360 градусов. Матлаб дополнительно имеет функцию atan2d с теми же аргументами, но возвращает результат в градусах (от -180 до +180).

Для ускорения вычислений также можно использовать и значения арктангенса, заданные таблично с требуемой точностью.

Обработка границ

Если рассмотреть амплитуды градиента яркости в различных точках, то можно заметить интересную особенность — он будет больше нуля не только в точках границы, но и в точках фона или объекта, которые границами не являются. Это происходит из-за небольшой разности освещенности соседних точек, из-за наличия у объекта текстуры и мелких деталей, из-за влияния шума и помех. Кроме того, ширина граничной области может составлять несколько пикселей, особенно для недостаточно резких границ. Тогда возникает задача отделить настоящие точки границ от остальных.

Цель этапа обработки границ — пометить точки изображения как принадлежащие либо не принадлежащие границам объекта (бинаризация границ) и уменьшить толщину контура до единицы.



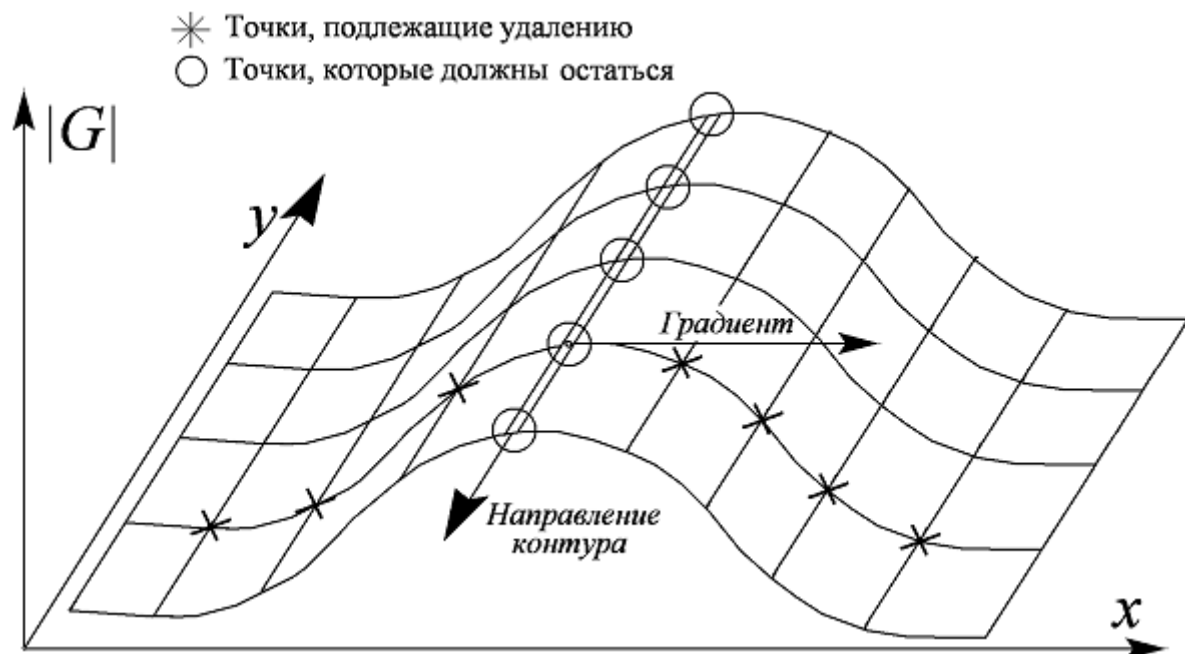
Уменьшение толщины границы

Простейший способ уменьшить толщину границы — применить морфологический алгоритм типа «пожар в лесу», который рассматривался ранее для получения лент. Однако такой алгоритм — многопроходный и работает достаточно медленно.

Более быстрый способ заключается в использовании принципа немаксимального подавления.

Рассмотрим сечение изображения, проходящее через анализируемую точку в направлении градиента яркости. Из рисунка видно, что если анализируемая точка является центральной точкой границы, то амплитуда градиента яркости образует локальный максимум: точки вокруг нее имеют меньшую амплитуду градиента. Если же точка не является центральной точкой, то в ее окрестности должны быть точки с большей амплитудой градиента. Такие точки необходимо удалять.

Однако этот принцип будет справедлив только в сечении градиента яркости. Если мы рассмотрим двухмерный профиль градиента яркости, то увидим, что в окрестностях центральной точки границы будут присутствовать как точки с меньшей амплитудой градиента (в направлении градиента), так и с примерно одинаковой или даже большей (в направлении контура). Таким образом, для поиска центральных точек границы необходимо анализировать только точки, лежащие в направлении градиента яркости.



Алгоритм немаксимального подавления можно записать следующим образом.

Для каждой точки изображения (x, y) , $x=2..ширина-2$, $y=2..высота-2$ с известным градиентом яркости G :

по углу направления градиента яркости выбираем соседние точки в сечении градиентом (например — берем их из таблицы)

если хоть в одной из этих соседних точек амплитуда градиента яркости больше, чем в текущей, то текущую точку помечаем как не содержащую границ (разметка выполняется в отдельном массиве)

если хоть в одной из этих соседних точек амплитуда градиента яркости равна амплитуде в текущей точке и соседняя точка лежит выше либо правее текущей, то текущую точку помечаем как не содержащую границ (разметка выполняется в отдельном массиве)

если ни одно из этих условий не выполнилось, текущую точку помечаем как точку границы.

Алгоритм может быть применен только к точкам, отстоящих на две точки от края изображения, поскольку для крайних точек нельзя посчитать градиент, а для их соседей нельзя применить алгоритм немаксимального подавления, поскольку для этого нужно знать градиент в крайних точках.

Разметка точек выполняется в отдельном массиве.

Для упрощения алгоритма список соседних точек для каждого направления градиента рассчитывается заранее и заносится в таблицу.

Алгоритм выполняется за один проход, не требует сложных вычислительных операций и выполняется достаточно быстро.

Выделение линий

Прежде всего необходимо отметить, что прямые линии на изображении могут проходить только через границы изображения. Внутри самого объекта, а также на фоне — линий, как правило, нет. Единственное исключение — случай, когда объект является протяженным (например, провода, треки частиц и т.д.). В этом случае прямая будет проходить не по границам, а параллельно им и между ними.

Таким образом, детектирование прямых всегда начинается с этапа получения и обработки точек границ, который мы рассмотрели выше.

Пусть имеется набор точек границы. Для каждой точки границы известны ее координаты и градиент яркости. Наша задача — найти прямые, проходящие через заданное количество точек границ.

Наиболее простой и очевидный алгоритм заключается в просмотре комбинаций пар точек. Каждая пара точек однозначно задает прямую, и после расчетов ее параметров легко определить, через какое количество точек пройдет прямая. Однако количество возможных пар точек на изображении очень велико (если точки контура составляют 13% точек изображения, то количество пар равно $0,017N^2$, где N — размер стороны изображения), и такой алгоритм будет вычислительно неэффективным. Желательно построить алгоритм, который обладал бы линейной сложностью относительно количества точек границ.

Выделение прямых при помощи алгоритма Хафа

Рассмотрим уравнение прямой, проходящей через точку границы $A(x_i, y_i)$:

$$y_i = k x_i + b$$

где k и b — параметры искомой прямой.

Пусть нас пока не смущает невозможность использования этого уравнения для описания вертикальных прямых — этот вопрос будет рассмотрен немного ниже.

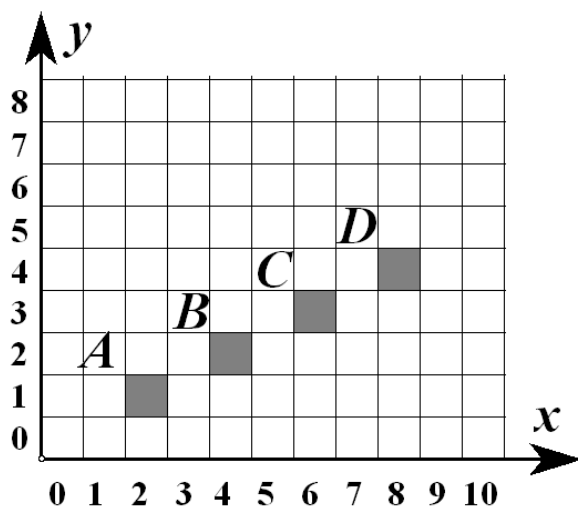
В этом уравнении x_i и y_i являются константами, а k и b являются неизвестными. Перепишем уравнение прямой в другом виде:

$$b = -x_i k + y_i$$

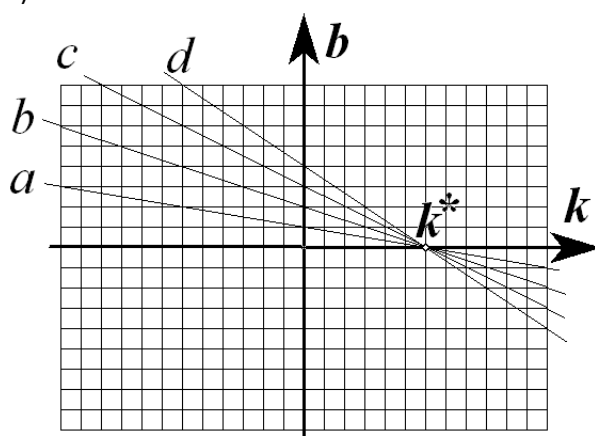
Легко убедиться, что это — тоже уравнение некоторой прямой, но в других координатах — k и b , а параметрами прямой являются $-x_i$ и y_i .

Введем в рассмотрение пространство параметров (ПП) прямой $\langle k, b \rangle$. В соответствии с приведенными выше формулами каждой точке пространства $\langle k, b \rangle$ очевидным образом соответствует прямая в пространстве координат изображения $\langle x, y \rangle$, а каждой точке границы $A(x_i, y_i)$ в пространстве координат изображения соответствует прямая в пространстве параметров прямой $\langle k, b \rangle$. Эти два пространства являются *дуальными* (рис. 3.1).

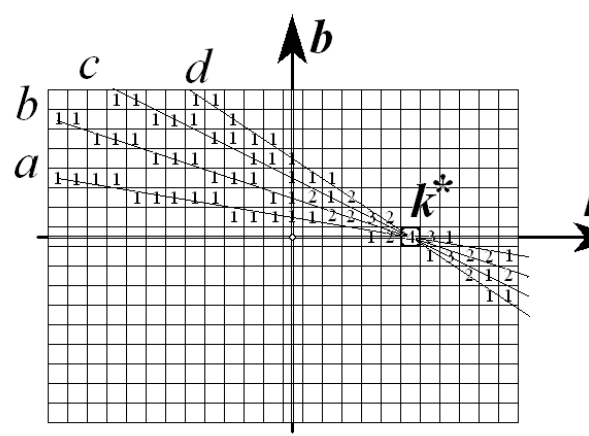
Предположим, что несколько точек границы лежат на одной прямой. Каждая из этих точек в ПП прямой будет порождать свою прямую. Однако все эти прямые должны пересечься в точке $\langle k^*, b^* \rangle$, где k^* и b^* — параметры прямой, проходящей через все рассматриваемые точки (рис. 3.1). Таким образом, для обнаружения прямых на изображении надо для каждой точки границы построить соответствующую прямую в ПП прямых, а затем найти точки пересечения существенного количества таких прямых.



а)



б)



в)

Рисунок 3.1 — Пространство параметров прямой

а) Изображение, содержащее четыре точки границы А,В,С и D, лежащие на одной прямой;

б) Пространство параметров прямой, содержащее прямые а,в,с,д, которые соответствуют точкам А,В,С, D;

в) Массив счетчиков, представляющий пространство параметров прямой. Максимальный элемент массива $M[k^*, b^*] = 4$ соответствует прямой $y = k^* x + b^*$, проходящей через точки А,В,С и D.

Для представления ПП прямых введем двумерный массив счетчиков (рис. 3.1,в) с индексами $[k, b]$. Каждая ячейка будет хранить количество прямых, проходящих через заданную точку (k, b) . Тогда алгоритм поиска прямых будет выглядеть следующим образом:

Алгоритм поиска прямых

1. Для каждой точки границы с координатами $A(x_i, y_i)$:

Строим прямую $b = -x_i k + y_i$ в пространстве $\langle k, b \rangle$. Для каждого возможного значения k вычисляем соответствующее значение b и увеличиваем счетчик $[k, b]$ на единицу.

2. Для каждой ячейки $[k, b]$ массива счетчиков: если значение счетчика $[k, b]$ больше порогового, то заносим прямую с параметрами k, b в список найденных прямых.

Под пороговым значением здесь понимается минимально допустимое количество точек границы, через которые проходит прямая. Оно может быть задано заранее либо рассчитано, например, таким образом, чтобы находить заданное количество прямых. Такой алгоритм называется преобразованием Хафа.

Недостатком алгоритма Хафа является невозможность выделения вертикальных линий: для них значение k убегает в бесконечность, а b не определено. Чтобы преодолеть этот недостаток, необходимо отдельно детектировать горизонтальные прямые (с углом наклона до 45 градусов) в пространстве $\langle x, y \rangle$, и отдельно — вертикальные (с углом наклона больше 45 градусов) в пространстве $\langle y, x \rangle$.

Такой алгоритм позволяет детектировать линии любой ориентации, однако он имеет другой недостаток: выделяемые линии оказываются неравноценными: точность описания линий с меньшим углом наклона выше, чем с большим. Пространство поиска $\langle k, b \rangle$ оказывается *анизотропным* — в нем различные направления неравноценны.

Использование нормальной формы прямой

Существует ли изотропное пространство параметров прямой? Безусловно. Для этого воспользуемся *нормальной формой* параметризации прямой. Нормальная форма предусматривает определение прямой через ее *вектор нормали* (ρ, φ) :

$$\rho = x \cos \varphi + y \sin \varphi$$

где ρ — длина нормали,

φ — угол наклона нормали,

x, y — координаты точки, принадлежащей прямой.

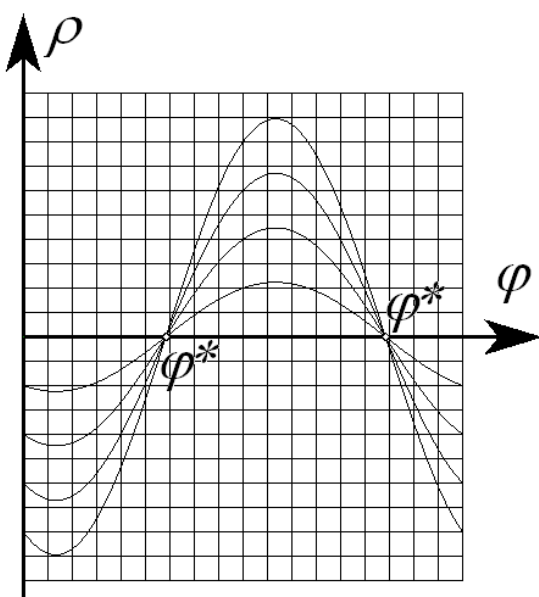
Вместо пространства $\langle k, b \rangle$ будем рассматривать пространство $\langle \rho, \varphi \rangle$. Алгоритм детектирования прямых изменится следующим образом:

Алгоритм детектирования прямых с использованием нормальной формы

1. Для каждой точки границы с координатами $A(x_i, y_i)$:

Строим синусоиду $\rho = x_i \cos \varphi + y_i \sin \varphi$ в пространстве $\langle \rho, \varphi \rangle$. Для φ от 0 до 360 градусов вычисляем соответствующее значение ρ и увеличиваем счетчик $H[\rho, \varphi]$ на единицу.

2. Для каждой ячейки $H[\rho, \varphi]$ массива счетчиков: если значение счетчика $H[\rho, \varphi]$ больше порогового, то заносим прямую с параметрами ρ, φ в список найденных прямых.



Данный алгоритм очень похож на предыдущий, но вместо прямых в пространстве параметров строятся синусоиды.

Одной прямой на изображении в пространстве $\langle \rho, \varphi \rangle$ будут соответствовать два максимума: (ρ, φ) и $(-\rho, \varphi + 180)$, один из которых следует исключить. Для этого следует либо отбрасывать отрицательные значения ρ , либо изменять φ от 0 до 180 градусов.

Максимальное значение $\rho_{\max}$ для верхней границы массива счетчиков найти достаточно легко: это максимально возможное расстояние до прямой на изображении. Расстояние будет максимальным, если прямая проходит через самый дальний угол изображения. Таким образом, $\rho_{\max}$ равно расстоянию до дальнего угла изображения.

Вычисление синусов и косинусов в данном алгоритме производится заранее, значения с нужным шагом переводятся в формат с фиксированной точкой и заносятся в таблицу.

Детектирование отрезков

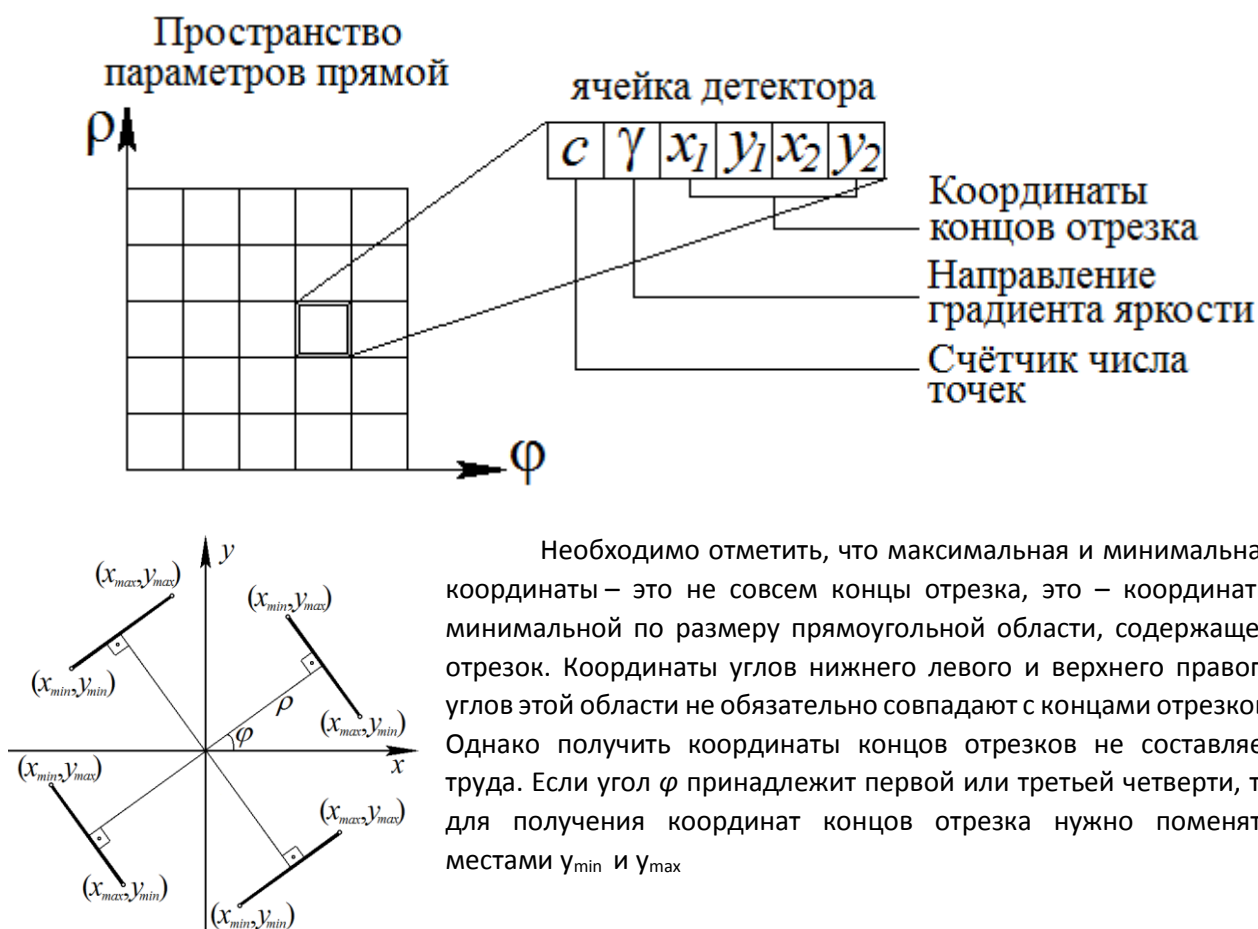
На практике гораздо более полезно иметь представление объекта не в виде прямых, а в виде отрезков: отрезки гораздо более локализованы в пространстве, и можно легко отделить отрезки, принадлежащие объекту интереса и не принадлежащие ему.

В случае с прямыми это сделать затруднительно.

Отрезок, в отличие от прямой, будет иметь уже четыре параметра вместо двух. Например, это могут быть координаты концов отрезка.

Предположим для начала, что на одной прямой может лежать только один отрезок. В этом упрощенном случае отрезок может быть однозначно идентифицирован прямой, проходящей через него. Тогда идентификационные параметры отрезка можно сократить с четырех до двух, например, до параметров нормали $\langle \rho, \varphi \rangle$

Что же делать с другими параметрами, необходимыми для полного описания отрезка? Мы отнесем их к дескриптивным параметрам. Для их учета и накопления расширим ячейки счетчика: пусть каждый счетчик теперь содержит не только количество точек в отрезке, но и дополнительную дескриптивную информацию, например – максимальную и минимальную координаты x и y отрезка. Получим *расширенное пространство поиска*: оно по-прежнему двумерно, но имеет более сложную структуру из-за учета дескриптивных параметров. Помимо максимальных и минимальных координат, расширенное пространство поиска может хранить и другую информацию, например, направление градиента вдоль отрезка.



Алгоритм поиска в расширенном пространстве поиска можно записать в следующем виде:

1. Для каждой точки границы $A(x_a, y_a)$ с градиентом $G(g_x, g_y)$

- рассчитываем $\langle \rho, \varphi \rangle$
- находим ячейку $H[\rho, \varphi]$
- увеличиваем счетчик: $H[\rho, \varphi].C++$
- корректируем концы отрезка:
 - если $H.x_{min} > x_a$ то $H.x_{min} = x_a$
 - если $H.x_{max} < x_a$ то $H.x_{max} = x_a$
 - если $H.y_{min} > y_a$ то $H.y_{min} = y_a$
 - если $H.y_{max} < y_a$ то $H.y_{max} = y_a$

2. Находим локальные максимумы $H^*[\rho^*, \varphi^*].C$, соответствующие наиболее длинным и четким отрезкам; если φ^* принадлежит первой или третьей четверти, то меняем местами φ_{min} и φ_{max} .

Очевидно, что алгоритм является модификацией алгоритма поиска прямых. Добавляются только операции сравнения концов, удельный вес которых в общей доле вычислений достаточно мал.

Ход работы

Часть 1. Поиск границ

Запустите графический редактор Paint (или любой другой) и нарисуйте черный пятиугольник. Сохраните изображение в формате BMP или PNG в рабочей папке со скриптом.

В Матлабе создайте новый скрипт, загрузите изображение с пятиугольником (imread), убедитесь, что оно загрузилось и может быть отображено на экране (imshow). Обратите внимание, что матрица изображения – трёхмерная, она содержит три цветовые плоскости. Преобразуем его в полутоновое серое (rgb2gray). Убедитесь, что матрица изображения стала двухмерной.

Будем считать, что яркость объекта нам неизвестна, и попробуем найти его границы.

Зададим маску фильтра Собела в виде матрицы 3х3: $S_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}$, $S_y = S_x'$ и найдем производные по обеим координатам (imfilter). Обратите внимание, что в процессе дифференцирования могут появляться отрицательные числа, которые не могут быть представлены в байтовом формате (uint8). Поэтому изображение нужно преобразовать в знаковый формат, например – double (im2double).

```
A=im2double(A);  
Dx=imfilter(A,Sx); %Производная по x даст вертикальные границы  
Dy=imfilter(A,Sy); %Производная по y даст вертикальные границы
```

Отобразите найденные границы на экране, убедитесь, что они отображены верно. Обратите внимание, что отрицательные производные (переход от белого к черному) отобразятся в виде черных точек, а положительные – в виде белых. Точки, не содержащие границ, отобразятся как серые.

Найдите модуль и фазу градиента яркости по формулам:

```
|G|=sqrt(Dx^2+Dy^2);  
γ=atan2d(Dy,Dx);
```

Отобразите модуль градиента на экране и рассмотрите его. Можно увидеть, что линии границ – достаточно толстые. Давайте уменьшим толщину границ до единицы (edge):

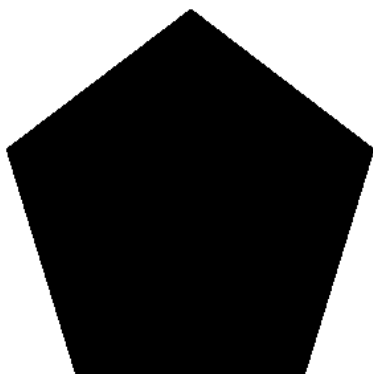
```
E=edge(A);
```

Внутри функции edge уже есть фильтр Собела и расчет градиента. Эта функция включает в себя все операции поиска границ, далее мы будем пользоваться в основном ею.

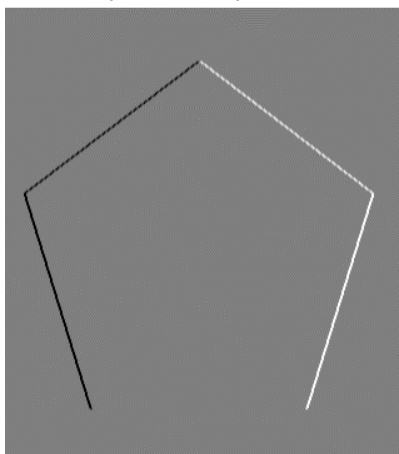
Далее приведен полный код первой части. Постарайтесь написать свой код и разобраться в особенностях используемых функций. Код приведен только для справки.

Результаты

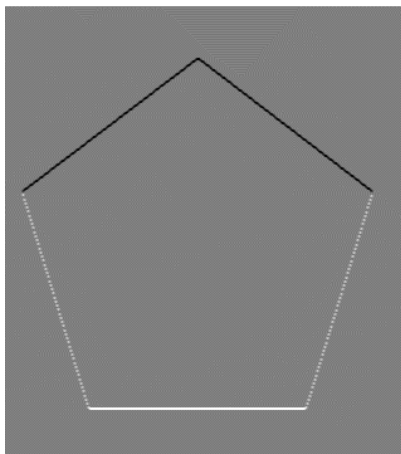
Загруженное изображение



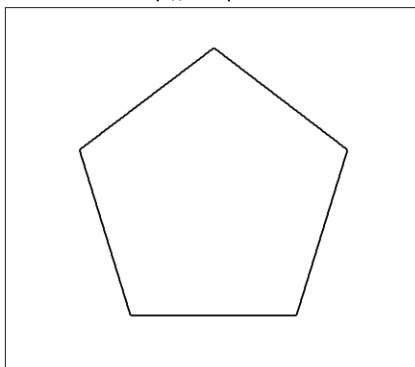
Вертикальные границы



Горизонтальные границы



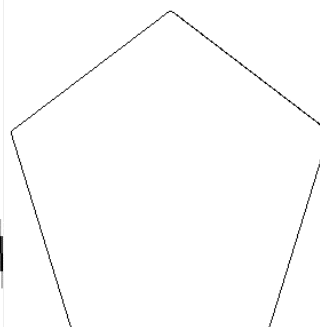
Градиент яркости



Градиент яркости



Границы единичной толщины



Часть 2. Поиск прямых с помощью преобразования Хафа

Создайте новый скрипт и скопируйте в него загрузку изображения пятиугольника и детектор границ. Это наша отправная точка.

Зададим пространство поиска в виде матрицы H размером 1000×1000 .

$KMAX=1000$;

$BMAX=1000$;

$H=zeros(KMAX,BMAX)$;

Зададим также диапазоны изменения параметров прямой на изображении:

$kmax=5$;

$bmax=1000$;

Это необходимо, поскольку параметры прямых на изображении могут изменяться в диапазоне плюс-минус бесконечности, а нам надо задать конечный диапазон, содержащий практически значимые прямые.

Для реализации преобразования Хафа нам потребуется преобразовывать параметры прямой k, b в индексы KI, BI матрицы H и обратно. Для этого можно воспользоваться формулами пересчета:

$KI=round((k+kmax)*(KMAX-1)/(2kmax)+1)$;

$BI=round((b+bmax)*(BMAX-1)/(2bmax)+1)$;

$k=(KI-1)*(2kmax)/(KMAX-1) - kmax$;

$b=(BI-1)*(2bmax)/(BMAX-1) - bmax$.

Множители $(KMAX-1)/(2kmax)$ и т.д. лучше просчитать заранее.

Теперь все готово для реализации поиска линий.

Реализуйте следующий алгоритм:

- Для каждой точки A , которая является точкой границы:
 - Перебираем все возможные индексы KI от 1 до $KMAX$,
 - Для KI вычисляем соответствующее k ,
 - Для k вычисляем $b=-k*x+y$;
 - Для b вычисляем индекс BI ,
 - Если индекс BI попадает в матрицу H ,
 - то увеличиваем элемент $H(KI, BI)$ на единицу.

Выведите матрицу H на экран в виде изображения.

Если все сделано правильно, на экране увидим четкие отчетливые отклики – «бабочки».

(Если не получилось – в первую очередь проверьте правильность индексации точек изображения: должно быть $A(y, x)$).

Отрисуем найденные линии.

Найдите положение (KI^*, BI^*) наибольшего отклика.

Пересчитайте индексы (KI^*, BI^*) в параметры (k^*, b^*) прямой на изображении.

Далее будем перебирать все возможные x , и для каждого из них рассчитывать $y=round(k*x+b)$.

Если пара (x, y) попадает на изображение A , то рисуем ее в виде черной точки: $A(x, y)=0$.

Отобразите найденную прямую и убедитесь, что прямая прошла точно через одну из сторон пятиугольника.

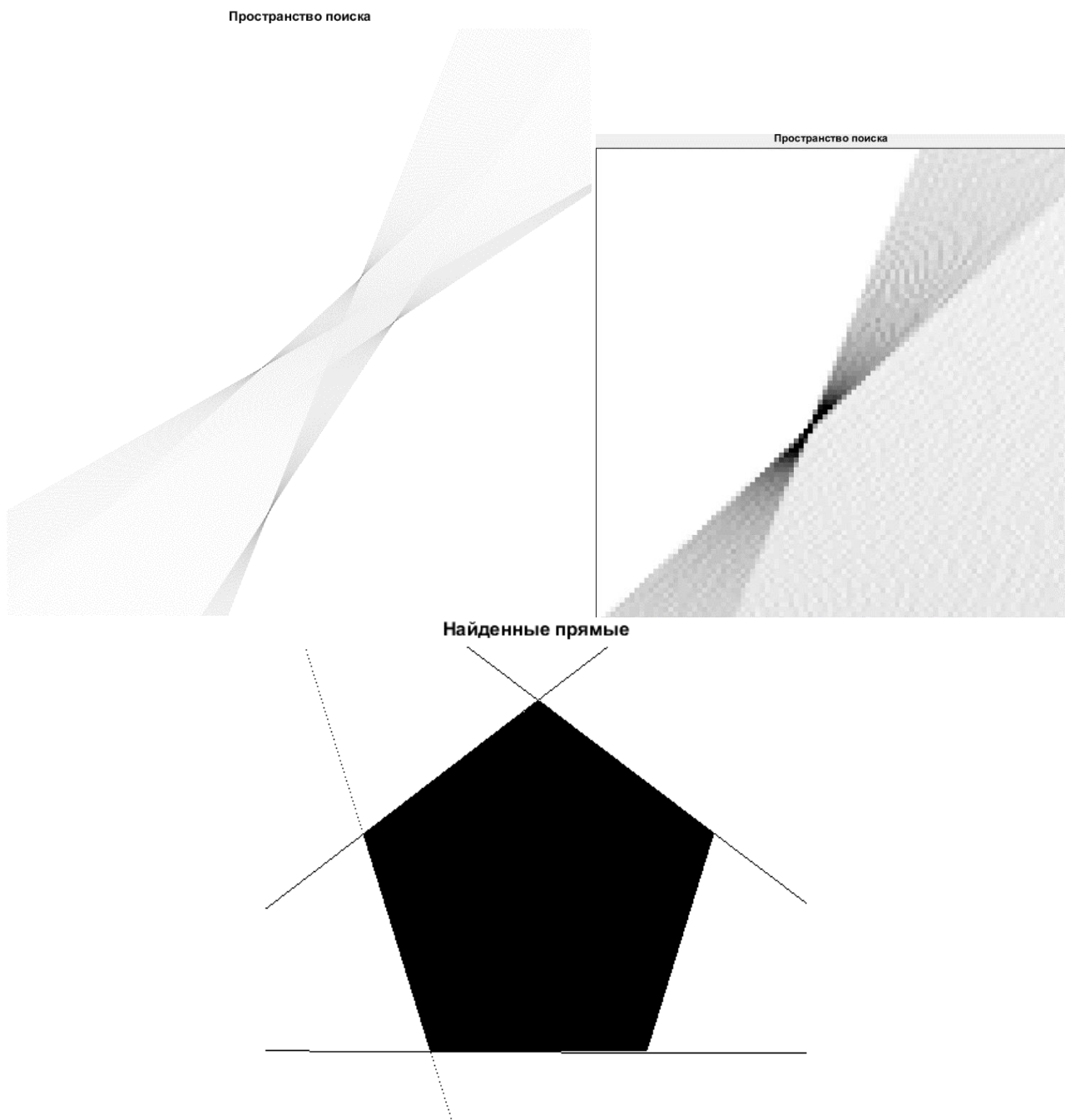
Мы получили только одну линию. Как найти остальные? Самый простой способ – удалить максимум из пространства поиска и повторить поиск (на практике так делать не нужно! Можно,

например, разбивать пространство на блоки, искать максимумы в каждом из блоков и сортировать по величине).

Обратите внимание, что максимум занимает несколько клеток. Поэтому удалить одну точку максимума недостаточно, нужно удалить целую область.

Удалим область максимума и повторим поиск и отрисовку прямой четыре раза. В результате получится пятиугольник с четырьмя найденными линиями.

Результаты



Часть 3. Поиск прямых в нормальной форме

В предыдущей части мы нашли только четыре прямых из пяти. Куда пропала пятая прямая? Дело в том, что стандартная форма прямой $y=kx+b$ не подходит для описания вертикальных прямых: для них k убегает в бесконечность, а b не определено. Более того, если прямая близка к вертикальной, параметры k и b становятся слишком большими и попросту не попадают в наше пространство H .

Как выйти из данной ситуации? Можно, конечно, повернуть изображение на 90 градусов и повторить поиск, однако по ряду причин это не самый лучший путь. Мы пойдем другим путем.

Запишем уравнение прямой в нормальной форме: $\rho = x \cos \varphi + y \sin \varphi$.

Здесь ρ – это длина нормали к прямой, а φ – ее угол наклона.

Тогда в дуальном пространстве будем получать не прямые, а синусоиды, и будем искать их точки пересечения.

Давайте реализуем этот метод.

Создайте новый скрипт, скопируйте загрузку изображения и поиск границ.

Зададим новое дуальное пространство. По координате φ оно будет иметь 360 отсчетов, по одному на градус, а по оси ρ размер равен диагонали изображения. Обратите внимание, что здесь нас будут интересовать только неотрицательные значения ρ .

```
[h,w]=size(A);  
RMAX=round(sqrt(w^2+h^2))+1;  
FMAX=360;  
H=zeros(RMAX,FMAX);
```

Теперь реализуем сам алгоритм:

- Для каждой точки A , которая является точкой границы:
 - Перебираем углы φ от 1 до 360,
 - Для φ вычисляем $\rho = x \cos \varphi + y \sin \varphi$,
 - Преобразуем ρ в целочисленный индекс $R = \text{round}(\rho) + 1$;
 - Если индекс R попадает в матрицу H ,
 - то увеличиваем элемент $H(R, \varphi)$ на единицу.

Выведите матрицу H на экран в виде изображения.

Если все сделано правильно, на экране увидим пучки синусоид, образующие пять отчетливых откликов в форме бабочек.

Отрисуем найденные линии.

Найдите положение (R^*, F^*) наибольшего отклика.

Пересчитайте индекс R в параметр ρ , вычитая единицу.

Далее будем перебирать все возможные x , и для каждого из них рассчитывать y . В соответствии с нормальной формой, $y = (\rho - x \cos \varphi) / \sin \varphi$.

Если пара (x, y) попадает на изображение A , то рисуем ее в виде черной точки: $A(x, y) = 0$.

Отобразите найденную прямую и убедитесь, что прямая прошла точно через одну из сторон пятиугольника.

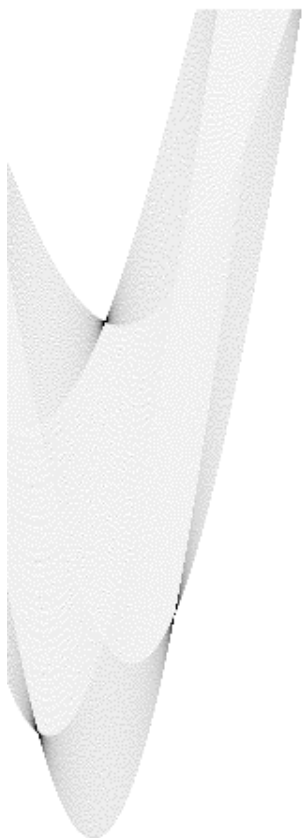
Чтобы найти остальные линии, удалим область максимума и повторим поиск и отрисовку прямой пять раз. В результате получится пятиугольник со всеми линиями.

Обратите внимание, что вертикальные линии получились очень тонкими. Дело в том, что для них значение φ мало, и небольшому шагу по x соответствует большой шаг по y . Что же делать?

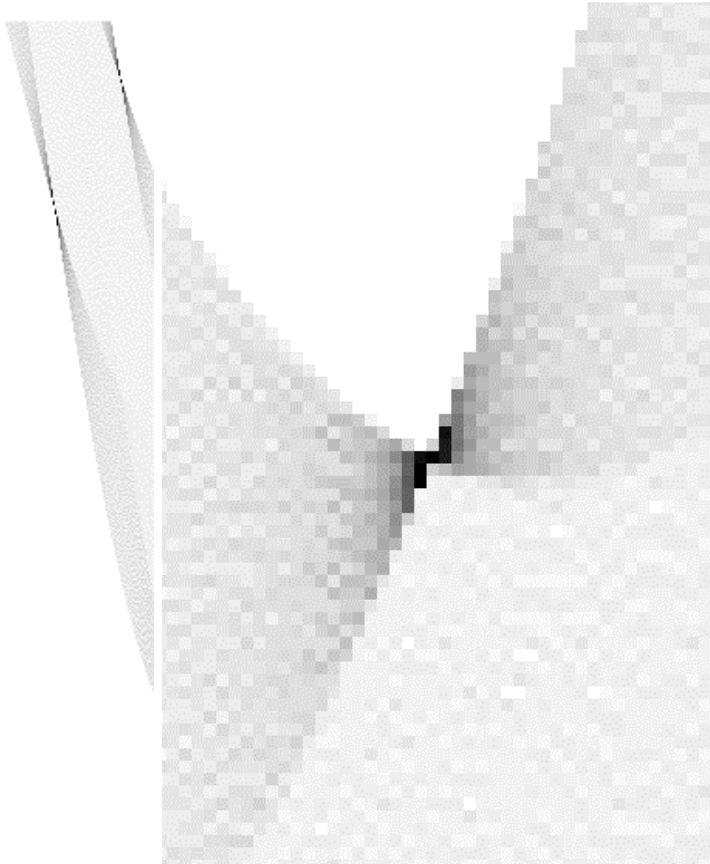
Ничего страшного. Для отрисовки вертикальных нужно перебирать не x , а y . Добавьте отрисовку вертикальных линий.

Результаты

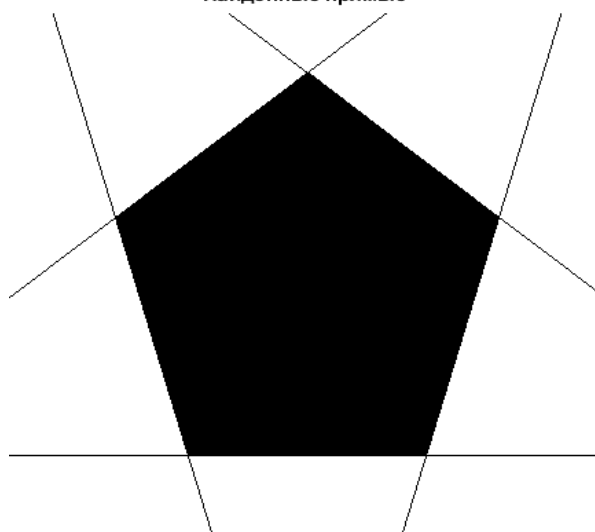
Пространство поиска



Пространство поиска



Найденные прямые



Часть 4. Поиск отрезков

В практических задачах нас будут интересовать не только и не столько прямые, сколько отрезки. Для поиска отрезков нам, помимо параметров прямой, потребуется найти также начало и конец отрезка. К счастью, преобразование Хафа позволяет это сделать достаточно просто.

Создайте новый скрипт, и скопируйте в него содержимое предыдущего скрипта.

Расширим пространство поиска, добавив в него плоскости для накопления максимальной и минимальной координаты:

```
Xmin=zeros(RMAX,FMAX);
Ymin=zeros(RMAX,FMAX);
Xmax=zeros(RMAX,FMAX);
Ymax=zeros(RMAX,FMAX);
```

Теперь будем накапливать не только голоса H , отданные за каждую прямую, но также и крайние точки прямых.

Скорректируем теперь цикл преобразования Хафа. После того, как мы увеличили счетчик $H(R, \phi)$, нужно дополнительно проверить – не является ли текущая точка максимальной или минимальной по x или по y . Тогда по окончании цикла у нас будет информация не только о количестве точек в каждой прямой, но также начальная и конечная точка на прямой, то есть отрезок.

Внесите в код эти изменения.

Теперь можно отобразить отрезки. Сделать это достаточно просто: теперь координата x (или y) в цикле отрисовки должна меняться не от 1 до ширины (высоты), а от минимальной координаты до максимальной.

Сделайте эти исправления, запустите код.

Куда исчезли наши линии?!

Мы нарисовали черные отрезки поверх черного пятиугольника, и их теперь не видно.

Чтобы исправить ситуацию, создадим новое чистое изображение B того же размера, что и исходное, и будем рисовать на нем.

Если все сделано правильно, мы увидим пять отрезков, образующих пятиугольник (отрезки могут не доходить до конца пятиугольника из-за скругленных краев и погрешности метода)

(Если не получилось, проверяйте поиск максимума и минимума в цикле, скорее всего ошиблись, копируя четыре одинаковые строчки).

Как проверить, правильно ли найдены отрезки?

Нарисуем найденные отрезки поверх пятиугольника, например – красным цветом. Для этого в красную плоскость добавим 255 на месте точек границ, а из синей и зеленой эти точки уберем:

```
B=uint8(B);
R=bitor(A,imcomplement(B));
G=bitand(A,B);
B=bitand(A,B);
```

Теперь склеим цветовые плоскости в одну трехмерную матрицу:

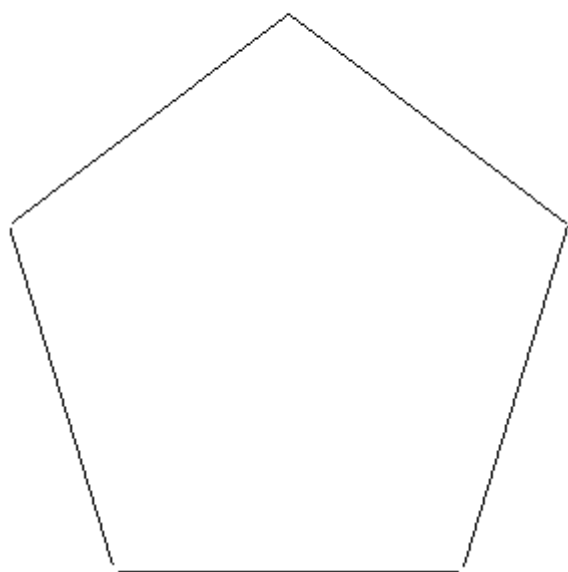
```
C=cat(3,R,G,B);
```

Отобразим ее на экране.

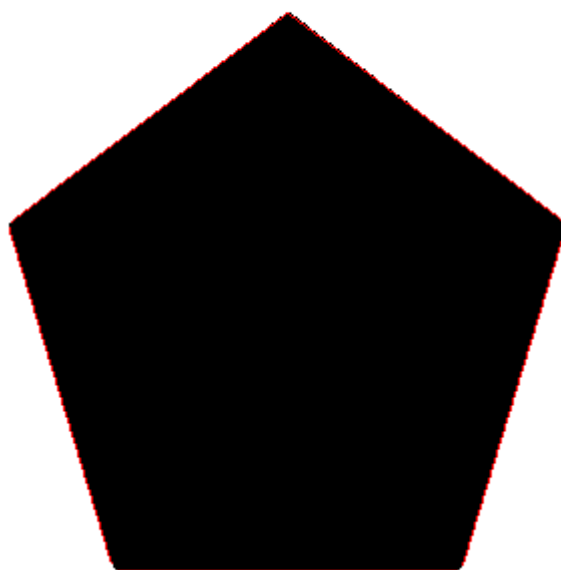
Видим, что все отрезки найдены верно.

Результаты

Найденные отрезки



Найденные отрезки



Вопросы и задания для самостоятельной работы

Ответы на вопросы даются в письменном виде и включаются в отчет.

Задания выполняются в письменном виде. Полученный результат и текст скрипта добавляются в отчет.

Часть 1

Сравните границы, которые находятся функциями:

```
E1=edge(A,'Sobel');
```

```
E2=edge(A,'Canny');
```

В чем отличие этих вызовов? Чем обусловлены отличия результата?

Часть 2

Почему в дуальном пространстве имеется только 4 максимума-бабочки? Куда делась еще одна? Как ее найти?

Что произойдет, если строить пятую линию, для которой отсутствует максимум в дуальном пространстве?

Ускорьте алгоритм, вынеся постоянные множители за пределы цикла. Используя функции tic и toc, определите прирост производительности.

Часть 3

Почему графики синусоид в дуальном пространстве отображены вверх ногами?

Модифицируйте алгоритм: перенесите начало координат в центр изображения. Убедитесь, что линии детектируются верно. Как изменится в этом случае дуальное пространство?

Ускорьте алгоритм: вынесите из цикла расчет синусов и косинусов (задайте таблицу синусов). Используя функции tic и toc, определите прирост производительности.

Часть 4

Ускорьте алгоритм, используя информацию о направлении градиента яркости. Погрешность определения градиента принять равной 10 градусов. Используя функции tic и toc, определите прирост производительности.

Приложения

Приложения содержат возможный вид программы (скрипта) для каждой из 4х частей лабораторной работы.

Используйте код только в ознакомительных целях. Постарайтесь написать свой код. Это нужно для того, чтобы мы с вами научились разрабатывать и поэтапно отлаживать большие фрагменты кода.

Код может содержать преднамеренные ошибки.

Приложение 1. Скрипт детектирования границ

```
clear;
close all;
A=imread('pentagon.bmp'); %Загружаем изображение пятиугольника

[h,w,p]=size(A);
if p==3, A=rgb2gray(A); end %Преобразуем цветное в полутоновое серое
figure,imshow(A),title('Загруженное изображение');

Sx=[-1 0 1;-2 0 2;-1 0 1]; %Маска фильтра Собела
Sy=Sx';

A=im2double(A);
Dx=imfilter(A,Sx); %Производная по x даст вертикальные границы
Dy=imfilter(A,Sy); %Производная по y даст вертикальные границы

figure,imshow(Dx,[],title('Вертикальные границы'));
figure,imshow(Dy,[],title('Горизонтальные границы'));

G=sqrt(Dx.^2+Dy.^2); %Модуль градиент яркости
gamma=atan2d(Dy,Dx); %Фаза градиента яркости
figure,imshow(imcomplement(G),[],title('Градиент яркости'));

E=edge(A); %Границы единичной толщины
figure,imshow(imcomplement(E)),title('Границы единичной толщины');
```

Приложение 2. Скрипт детектирования линий в стандартной форме

```
% -----
% Пример кода для л/р №3
% "Обработка границ, детектирование прямых и отрезков"
% Часть 2 - поиск прямых в стандартной форме

clear;
close all;
A=imread('pentagon.bmp'); %Загружаем изображение пятиугольника

[h,w,p]=size(A);
if p==3, A=rgb2gray(A);end %Преобразуем цветное в полутоновое серое
% figure,imshow(A),title('Загруженное изображение'); %только для отладки

E=edge(A); %Границы единичной толщины
% figure,imshow(imcomplement(E)),title('Границы единичной толщины'); %только для отладки

%Пространство поиска
KMAX=1000;
BMAX=1000;
H=zeros(KMAX,BMAX);

%Ограничения на линии
kmax=5;
bmax=1000;

%Размеры изображения
[h,w]=size(A);

%Преобразование Хафа
for x=1:w
    for y=1:h
        if E(y,x)==1
            for KI=1:KMAX
                k=(KI-1)*(2*kmax)/(KMAX-1)-kmax;
                b=-x*k+y;
                BI=round((b+bmax)*(BMAX-1)/(2*bmax)+1);
                if (BI>=1) && (BI<=BMAX)
                    H(KI,BI)=H(KI,BI)+1;
                end
            end
        end
    end
end

figure,imshow(-H,[],title('Пространство поиска'));

for Pass=1:4

    %Ищем максимальный отклик
    Hbest=max(H(:));
    [KIbest,BIbest]=find(H==Hbest);
    KIbest=KIbest(1);
    BIbest=BIbest(1);

    kbest=(KIbest-1)*(2*kmax)/(KMAX-1)-kmax;
    bbest=(BIbest-1)*(2*bmax)/(BMAX-1)-bmax;

    %Рисуем прямую
```

```
for x=1:w
    y=round(kbest*x+bbest);
    if (y>=1) && (y<=h)
        A(y,x)=0;
    end
end

%Ставим заплатку поверх найденного максимума
d=5;
H(KIbest-d:KIbest+d,BIbest-d:BIbest+d)=0;

end

figure,imshow(A),title('Найденные прямые');
```

Приложение 3. Скрипт детектирования линий в нормальной форме

```
% -----
% Пример кода для л/р №3
% "Обработка границ, детектирование прямых и отрезков"
% Часть 3 - поиск прямых в нормальной форме

clear;
close all;
A=imread('pentagon.bmp'); %Загружаем изображение пятиугольника

[h,w,p]=size(A);
if p==3, A=rgb2gray(A);end %Преобразуем цветное в полутоновое серое
% figure,imshow(A),title('Загруженное изображение'); %только для отладки

E=edge(A); %Границы единичной толщины
% figure,imshow(imcomplement(E)),title('Границы единичной толщины'); %только для отладки

[h,w]=size(A);
RMAX=round(sqrt(w^2+h^2))+1;
FMAX=360;
H=zeros(RMAX,FMAX);

%Преобразование Хафа в нормальной форме
for x=1:w
    for y=1:h
        if E(y,x)==1
            for f=1:360
                ro=x*cosd(f)+y*sind(f);
                R=round(ro)+1;
                if (R>=1) && (R<=RMAX)
                    H(R,f)=H(R,f)+1;
                end
            end
        end
    end
end

figure,imshow(-H,[],),title('Пространство поиска');

for pass=1:5

    Hmax=max(H(:));
    [Rbest,Fbest]=find(H==Hmax);
    Rbest=Rbest(1);
    Fbest=Fbest(1);
    ro=Rbest-1;

    if abs(sind(Fbest))>1/sqrt(2)
        for x=1:w
            y=(ro-x*cosd(Fbest))/sind(Fbest);
            y=round(y);
            if (y>=1) && (y<=h)
                A(y,x)=0;
            end
        end
    else
        for y=1:h
            x=(ro-y*sind(Fbest))/cosd(Fbest);
            x=round(x);
            if (x>=1) && (x<=w)
                A(y,x)=0;
            end
        end
    end
end
```

```
        end
    end
end

d=5;
H(Rbest-d:Rbest+d,Fbest-d:Fbest+5)=0;

end

figure,imshow(A),title('Найденные прямые');
```

Приложение 4. Скрипт детектирования отрезков

```
% -----
% Пример кода для л/р №3
% "Обработка границ, детектирование прямых и отрезков"
% Часть 4 - поиск отрезков

clear;
close all;
A=imread('pentagon.bmp'); %Загружаем изображение пятиугольника

[h,w,p]=size(A);
if p==3, A=rgb2gray(A);end %Преобразуем цветное в полутоновое серое
E=edge(A); %Границы единичной толщины

[h,w]=size(A);
RMAX=round(sqrt(w^2+h^2))+1;
FMAX=360;
H=zeros(RMAX,FMAX);
%Расширенное пространство
Xmin=H+w;
Ymin=H+h;
Xmax=H;
Ymax=H;

%Преобразование Хафа в нормальной форме
for x=1:w
    for y=1:h
        if E(y,x)==1
            for f=1:360
                ro=x*cosd(f)+y*sind(f);
                R=round(ro)+1;
                if (R>=1) && (R<=RMAX)
                    H(R,f)=H(R,f)+1;
                    %Здесь добавляем поиск крайних координат
                    Xmin(R,f)=min(x,Xmin(R,f));
                    Ymin(R,f)=min(y,Ymin(R,f));
                    Xmax(R,f)=max(x,Xmax(R,f));
                    Ymax(R,f)=max(y,Ymax(R,f));
                end
            end
        end
    end
end

figure,imshow(-H,[],),title('Пространство поиска');

B=zeros(size(A))+255; %вспомогательное изображение для отрисовки отрезков

for pass=1:5

    Hmax=max(H(:));
    [Rbest,Fbest]=find(H==Hmax);
    Rbest=Rbest(1);
    Fbest=Fbest(1);
    ro=Rbest-1;

    if abs(sind(Fbest))>1/sqrt(2)
        for x=Xmin(Rbest,Fbest):Xmax(Rbest,Fbest) %Здесь изменим границы цикла
            y=(ro-x*cosd(Fbest))/sind(Fbest);
            y=round(y);
```

```

        if (y>=1) && (y<=h)
            B(y,x)=0; %Здесь рисуем на вспомогательном изображении
        end
    end
else
    for y=Ymin(Rbest,Fbest):Ymax(Rbest,Fbest) %Здесь изменим границы цикла
        x=(ro-y*sind(Fbest))/cosd(Fbest);
        x=round(x);
        if (x>=1) && (x<=w)
            B(y,x)=0; %Здесь рисуем на вспомогательном изображении
        end
    end
end

d=5;
H(Rbest-d:Rbest+d,Fbest-d:Fbest+5)=0;

end

figure,imshow(B),title('Найденные отрезки');

%Отметим найденные отрезки красным цветом
B=uint8(B); %функции bitor/bitand принимают аргументы только одинакового типа
R=bitor(A,imcomplement(B)); %красная плоскость - добавим точки отрезка
G=bitand(A,B); %зеленая - уберем
B=bitand(A,B); %синяя - тоже уберем
C=cat(3,R,G,B); %склеим плоскости в одну матрицу
figure,imshow(C),title('Найденные отрезки');

```